The FitZone Membership Desk

Week 8 Coding Assignment - Question 4

Design: Member 1--0..* Membership; Membership 1--1 MembershipPlan. Plans calculate fees; Membership owns and validates status transitions Active, Frozen, Expired.
Java solution:

```
interface MembershipPlan { String name(); int months(); double discount(); default double calculateFee(){return 1000.0*months()*(1-discount());} }
class Monthly implements MembershipPlan {public String name(){return "Monthly";}public int months(){return 1;}public double discount(){return 0;}}
class Quarterly implements MembershipPlan {public String name(){return "Quarterly";}public int months(){return 3;}public double discount(){return .10;}}
class Annual implements MembershipPlan {public String name(){return "Annual";}public int months(){return 12;}public double discount(){return .25;}}
class Member {final String name;Membership membership;Member(String n){name=n;}}
class Membership {
enum Status {Active,Frozen,Expired} final Member member;final MembershipPlan plan;private Status status=Status.Active;
Membership(Member m,MembershipPlan p){member=m;plan=p;} Status status(){return status;}
boolean checkIn(){return status==Status.Active;}
void freeze(){if(status==Status.Expired)throw new IllegalStateException("Cannot freeze an Expired membership.");if(status!=Status.Active)throw new IllegalStateException("Only Active membership can be frozen.");status=Status.Frozen;}
void unfreeze(){if(status==Status.Expired)throw new IllegalStateException("Cannot unfreeze an Expired membership.");if(status!=Status.Frozen)throw new IllegalStateException("Only Frozen membership can be unfrozen.");status=Status.Active;}
void expire(){status=Status.Expired;}
}
public class Main {public static void main(String[] args){
Member a=new Member("Asha"),r=new Member("Ravi");a.membership=new Membership(a,new Quarterly());r.membership=new Membership(r,new Monthly());
System.out.printf("Quarterly membership created for Asha. Fee: Rs %.2f. Status: %s.%n",a.membership.plan.calculateFee(),a.membership.status());
System.out.printf("Monthly membership created for Ravi. Fee: Rs %.2f. Status: %s.%n",r.membership.plan.calculateFee(),r.membership.status());
if(a.membership.checkIn())System.out.println("Asha checked in successfully.");a.membership.freeze();System.out.println("Asha’s membership frozen. Status: Frozen.");
if(!a.membership.checkIn())System.out.println("Check-in denied: Asha’s membership is Frozen.");
r.membership.expire();System.out.println("Ravi’s membership expired. Status: Expired.");
try{r.membership.freeze();}catch(IllegalStateException e){System.out.println(e.getMessage());}
} }
```

Expected fees: Quarterly Rs 2700.00; Monthly Rs 1000.00. Only Active memberships may check in. To add Half-Yearly, implement MembershipPlan; no workflow change is needed.